

Lesson 9: Introduction to Databases and SQL

Python Essentials — Code the Dream

Game Plan for Today

Section	Time	What We'll Do
Warm-Up	5 min	Design a schema on paper
I Do	10–15 min	SQL basics, CREATE, INSERT, SELECT, JOINS
We Do	15–20 min	Build a school database together
You Do	5–10 min	Write SQL queries
Wrap-Up	5 min	Assignment preview & capstone DB goals

Warm-Up (5 min)

Design a Schema

A library has books and members. Members can borrow books.

Quick questions:

1. What tables would you need?
2. What columns would each table have?
3. Can a member borrow multiple books? Can a book be borrowed by multiple members?

Sketch your answer — share in the chat.

I Do: Why SQL?

SQL databases store structured, related data in tables.

Feature	CSV File	SQL Database
Updated in real-time?	No — static snapshot	Yes
Relationships?	No	Yes — foreign keys
Search/filter?	Pandas (in memory)	SQL queries (efficient)
Data integrity?	None	Constraints enforce rules
Multiple users?	Conflicts	Transactions keep it safe

We'll use SQLite — a file-based database built into Python:

```
import sqlite3
conn = sqlite3.connect("school.db")
```

I Do: Creating Tables

CREATE TABLE defines the structure:

```
CREATE TABLE IF NOT EXISTS Students (  
    student_id INTEGER PRIMARY KEY,  
    name TEXT NOT NULL UNIQUE,  
    age INTEGER,  
    major TEXT  
);
```

Key constraints:

Constraint	What It Does
PRIMARY KEY	Unique identifier, auto-generated
NOT NULL	Column must have a value
UNIQUE	No duplicate values allowed

I Do: Relationships and Join Tables

One-to-many: One publisher → many magazines

```
CREATE TABLE Magazines (  
    magazine_id INTEGER PRIMARY KEY,  
    name TEXT NOT NULL UNIQUE,  
    publisher_id INTEGER,  
    FOREIGN KEY (publisher_id) REFERENCES Publishers(publisher_id)  
);
```

Many-to-many: Students ↔ Courses (need a **join table**)

I Do: Relationships and Join Tables (Cont.)

```
CREATE TABLE Enrollments (  
    enrollment_id INTEGER PRIMARY KEY,  
    student_id INTEGER,  
    course_id INTEGER,  
    FOREIGN KEY (student_id) REFERENCES Students(student_id),  
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)  
);
```

A join table sits in the middle and holds two foreign keys.

I Do: Inserting Data

INSERT INTO adds records:

```
cursor.execute(  
    "INSERT INTO Students (name, age, major) VALUES (?, ?, ?)",  
    ("Alice", 20, "Computer Science")  
)  
conn.commit() # Don't forget to commit!
```

Key points:

- Use **parameterized queries** (?) — never string concatenation
- **PRIMARY KEY** is auto-generated if you don't specify it
- `conn.commit()` makes the changes permanent
- Without commit, changes are **rolled back** when the connection closes

I Do: Querying Data — SELECT

SELECT retrieves data:

```
-- All students
SELECT * FROM Students;

-- Specific columns
SELECT name, age FROM Students;

-- With a filter
SELECT * FROM Students WHERE age > 21;

-- Sorted
SELECT * FROM Students ORDER BY name;
```

I Do: Querying Data — SELECT (Cont.)

```
-- Combined conditions  
SELECT * FROM Students WHERE age > 20 AND major = 'Biology';
```

In Python:

```
cursor.execute("SELECT * FROM Students WHERE name = ?", ("Alice",))  
results = cursor.fetchall()  
for row in results:  
    print(row) # Each row is a tuple
```

I Do: JOINS — Combining Tables

JOINS combine related tables:

```
SELECT s.name, c.course_name
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
JOIN Courses c ON e.course_id = c.course_id;
```

Join Type	What It Returns
JOIN (inner)	Only matching rows
LEFT JOIN	All left rows + matches from right
RIGHT JOIN	All right rows + matches from left
FULL JOIN	Everything from both tables

LEFT JOIN is useful when you want ALL records from one table even if there's no

I Do: UPDATE and DELETE

Modify existing records:

```
-- Update a student's name and age
UPDATE Students SET name = 'Charles', age = 20
WHERE name = 'Charlie';

-- Raise all salaries by 10%
UPDATE products SET price = price * 1.1;
```

Remove records:

```
-- Delete products under $1
DELETE FROM products WHERE price < 1.0;
```

Warning: Without a `WHERE` clause:

- `UPDATE` changes **every** row
- `DELETE` removes **every** row

I Do: SQL from Pandas

Load SQL query results directly into a DataFrame:

```
import pandas as pd
import sqlite3

with sqlite3.connect("lesson.db") as conn:
    sql = """
    SELECT c.customer_name, p.product_name, li.quantity
    FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    JOIN line_items li ON o.order_id = li.order_id
    JOIN products p ON li.product_id = p.product_id
    """
    df = pd.read_sql_query(sql, conn)
    print(df.head())
```

This is the bridge between SQL and everything you've learned about Pandas. Query with SQL, analyze with Pandas.

We Do: Build the School Database

Let's build the Students/Courses/Enrollments schema together.

Step 1: Connect and create tables:

```
import sqlite3

with sqlite3.connect("school.db") as conn:
    conn.execute("PRAGMA foreign_keys = 1")
    cursor = conn.cursor()

    cursor.execute("""
CREATE TABLE IF NOT EXISTS Students (
    student_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL UNIQUE,
    age INTEGER,
    major TEXT
)""")
```

We Do: Build the School Database (Cont.)

```
cursor.execute("""  
CREATE TABLE IF NOT EXISTS Courses (  
    course_id INTEGER PRIMARY KEY,  
    course_name TEXT NOT NULL UNIQUE,  
    instructor_name TEXT  
)""")
```

What constraint prevents duplicate student names?

We Do: Insert Data Safely

Step 2: Create a reusable function with error handling:

```
def add_student(cursor, name, age, major):  
    try:  
        cursor.execute(  
            "INSERT INTO Students (name, age, major) VALUES (?, ?, ?)",  
            (name, age, major)  
        )  
    except sqlite3.IntegrityError:  
        print(f"{name} is already in the database.")  
  
add_student(cursor, "Alice", 20, "Computer Science")  
add_student(cursor, "Bob", 22, "History")  
add_student(cursor, "Charlie", 19, "Biology")  
conn.commit()
```

Why do we catch `IntegrityError`?

The UNIQUE constraint on `name` raises this error if we try to add a duplicate.

We Do: Enroll Students in Courses

Step 3: Create enrollments using foreign keys:

```
def enroll_student(cursor, student_name, course_name):  
    cursor.execute(  
        "SELECT student_id FROM Students WHERE name = ?",  
        (student_name,))  
    student = cursor.fetchall()  
  
    cursor.execute(  
        "SELECT course_id FROM Courses WHERE course_name = ?",  
        (course_name,))  
    course = cursor.fetchall()
```

We Do: Enroll Students in Courses (Cont.)

```
if student and course:
    cursor.execute(
        "INSERT INTO Enrollments (student_id, course_id) VALUES (?,?)",
        (student[0][0], course[0][0]))

enroll_student(cursor, "Alice", "Math 101")
conn.commit()
```

Key insight: We look up the IDs first, then use them as foreign keys.

We Do: Query with JOINS

Step 4: Find which students are in which courses:

```
cursor.execute("""
    SELECT s.name, c.course_name
    FROM Students s
    JOIN Enrollments e ON s.student_id = e.student_id
    JOIN Courses c ON e.course_id = c.course_id
""")

for row in cursor.fetchall():
    print(f"{row[0]} is enrolled in {row[1]}")
```

We Do: Query with JOINS (Cont.)

Output:

```
Alice is enrolled in Math 101  
Alice is enrolled in Chemistry 101  
Bob is enrolled in English 101  
...
```

Discussion: Why do we need TWO JOINS here?

Because Students and Courses aren't directly linked — Enrollments sits in the middle.

You Do: SQL Query Challenge (5 min)

Using the school database we just built, write SQL for:

1. Find all students majoring in "Biology"

Hint: `SELECT * FROM Students WHERE ...`

2. Find all courses taught by "Dr. Smith"

Hint: `SELECT * FROM Courses WHERE ...`

3. Count how many students are enrolled in each course

Hint: `SELECT c.course_name, COUNT(*) FROM ... GROUP BY ...`

Assignment Preview

Task	What You'll Do
1	Create a SQLite database (<code>magazines.db</code>)
2	Design schema: publishers, magazines, subscribers, subscriptions
3	Write functions to populate all 4 tables
4	Write SQL queries with JOINS
5	Load <code>lesson.db</code> into Pandas, groupby, save CSV
6	Store capstone scraping data in SQLite

Assignment Tips

Task 2 (schema design): Think about relationships first:

- Publishers → Magazines: one-to-many (foreign key in `magazines`)
- Subscribers ↔ Magazines: many-to-many (join table = `subscriptions`)

Task 3 (populating): Watch for duplicates! The subscriber check is trickier — same name AND same address means duplicate.

Task 5 (Pandas + SQL): Use `pd.read_sql_query()` to load, then it's regular Pandas (groupby, agg, sort, to_csv).

If you mess up the database: Just delete `magazines.db` and re-run your script.

Wrap-Up

Today we covered:

- Why SQL databases vs flat files
- Creating tables with constraints (PRIMARY KEY, NOT NULL, UNIQUE, FOREIGN KEY)
- One-to-many and many-to-many relationships (join tables)
- INSERT with parameterized queries
- SELECT with WHERE, ORDER BY, and JOINS
- UPDATE and DELETE
- Loading SQL results into Pandas DataFrames

Quick check: What's the difference between a primary key and a foreign key?

Getting Help This Week

- Install the **SQLite Viewer** VS Code extension to inspect your database
- Use `sqlcommand.py` from the lesson to practice queries interactively
- If your database gets messy, **delete the .db file** and re-run
- SQLBolt tutorial (optional but recommended): sqlbolt.com
- Post questions in **#discussion** on Slack

Next week: Advanced SQL — subqueries, aggregation, window functions, and more!

Data, Meet Database!

Structured storage for structured insights.

See you next session!